

Attendance & Workforce Admin

Phase 1 — Complete Technical Documentation

Scope Frontend on mock data. No backend, no database.
Stack React 18 · TypeScript · Vite · Tailwind · TanStack Query · MSW
Size 84 source files · ~11,150 lines
Status Complete. Ready for Phase 2.
Date September 2026

Contents

- 1 What was built
- 2 High-level design
- 3 Low-level design — the six mechanisms that matter
- 4 Complete file reference
- 5 Conventions — the rules that keep it coherent
- 6 Phase 2 migration map

1. What was built

A workforce admin dashboard covering attendance, tasks and access control, running entirely on mock data served through a Service Worker. Every screen is finished, every endpoint has a handler, and the interface behaves exactly as it will once a real server answers — because the swap is one line.

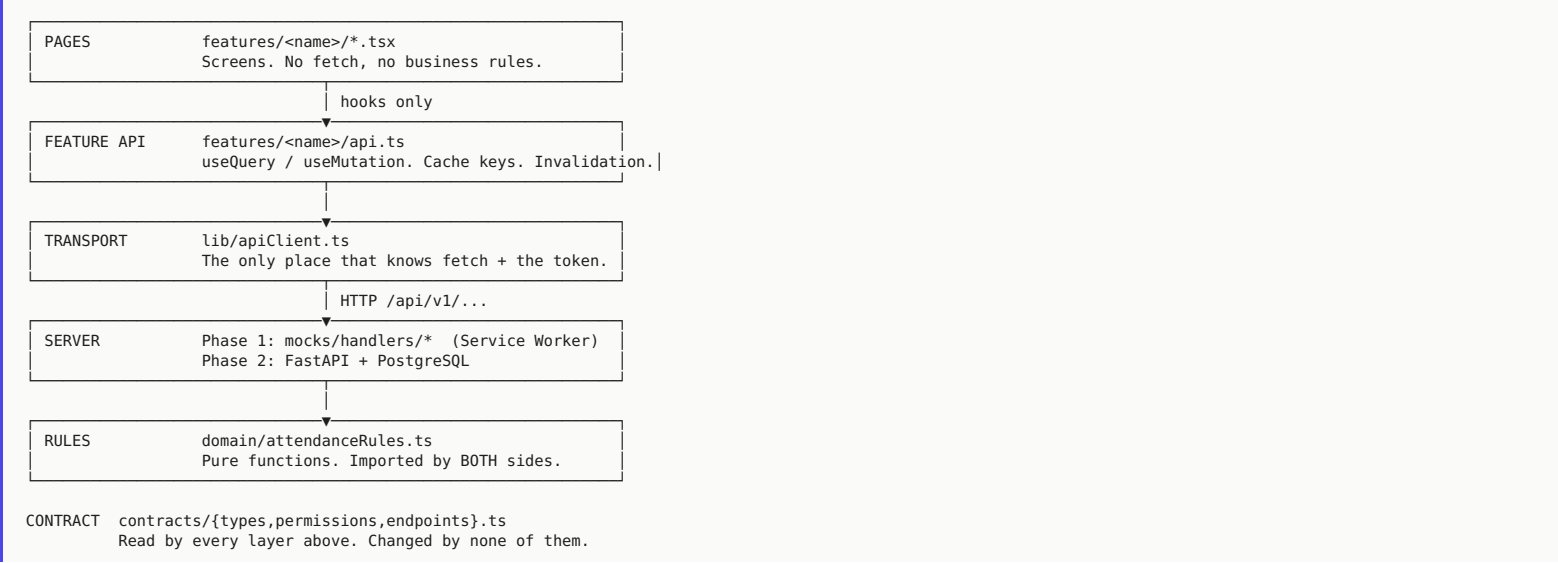
Module	What it does	Who sees it
Dashboard	Own check-in strip; company counters, live present list, exception panel, attendance trend chart; or personal tasks and hours chart	Everyone, content by capability
Employees	Directory, create, edit, detail drawer with attendance and task tabs	<code>employees.view_all</code>
Attendance	Daily register, date-range history, raw punch drill-down, manual punch entry	All — scoped to own or everyone
Tasks	Three-column board, assign, self-add, edit, start, complete, reopen	All — scoped
Roles & Permissions	Permission matrix with staged edits, create role, plain-English guide	<code>roles.manage</code>
Access Control	Login accounts, create, reset password, enable/disable, change role	<code>users.manage</code>
Settings	Own profile, own password, company attendance policy	Everyone; policy gated

Cross-cutting features

Feature	Where it lives
Permission system	<code>contracts/permissions.ts</code> , <code>lib/rbac.ts</code> , <code>routes/ProtectedRoute.tsx</code>
Session limits	<code>lib/session.ts</code> , <code>lib/useIdleTimer.ts</code> , <code>layouts/SessionGuard.tsx</code>
Confirm dialogs and toasts	<code>components/common/ConfirmDialog.tsx</code> , <code>Toast.tsx</code>
Responsive shell	<code>stores/uiStore.ts</code> , <code>layouts/Sidebar.tsx</code> , <code>AppShell.tsx</code> , <code>MobileTopBar.tsx</code>
Attendance rules	<code>domain/attendanceRules.ts</code> — the only file with business logic in it

2. High-level design

2.1 Layer diagram



THE SINGLE IDEA THE WHOLE ARCHITECTURE RESTS ON
Components call hooks. Hooks call `apiClient`. `apiClient` calls a URL. Whatever answers that URL is not the application's concern. Phase 1 it is a Service Worker; Phase 2 it is FastAPI. Because no component and no hook ever learns which, the cutover is deleting one call in `main.tsx`.

2.2 Request lifecycle, end to end



2.3 State ownership — four kinds, four homes

Kind	Lives in	Examples
Server state	TanStack Query cache	Employees, attendance days, tasks, roles, trend
Session state	stores/authStore.ts	Current user, permission Set, logout reason
Chrome state	stores/uiStore.ts	Sidebar collapsed, mobile drawer open
Filter state	The URL query string	Search text, department, date range, page

Filters live in the URL so the back button undoes a filter, a filtered view is shareable as a link, and a refresh does not lose the work. Holding them in `useState` loses all three, and retrofitting means rewriting every filter handler.

2.4 Routing



Every page except login is lazy-loaded, so a person who only opens the dashboard never downloads the roles matrix. Login is eager, because deferring the first thing everyone sees would add a spinner to the first paint.

3. Low-level design — the six mechanisms that matter

3.1 Attendance rollup

src/domain/attendanceRules.ts

The heart of the system. Punch events are raw facts; a "day" is a view computed from them, never stored.

```
buildDay({ employee, date, punches, shift }) → AttendanceDay

  sort punches by timestamp
  ins  = punches where direction = IN
  outs = punches where direction = OUT

  firstIn = ins[0]
  lastOut = outs[last]

  if no punches:
    status = WEEKEND if weekend else ABSENT
    return with zero minutes

  workedMinutes:
    firstIn AND lastOut → difference between them
    firstIn, no lastOut, today → difference to now
    otherwise           → 0   (never invent a departure time)

  flags (zero or more):
    LATE_IN      firstIn later than shiftStart + graceMinutes
    MISSING_OUT  firstIn, no lastOut, and the day is past
    MISSING_IN   lastOut with no firstIn
    EARLY_OUT    lastOut more than 15 min before shiftEnd
    SHORT_HOURS  workedMinutes below shift.minHours
    MANUAL_ENTRY any punch has source 'manual'

  status (exactly one): WEEKEND | PRESENT | ABSENT
```

Why first-in / last-out, not strict pairing

Strict IN/OUT/IN/OUT pairing sounds more correct and is far more fragile: one missed punch in the middle cascades and corrupts the rest of the day. First-and-last degrades gracefully — somebody who steps out for lunch and punches four times still has one arrival and one departure.

Why status and flags are separate fields

status answers "what kind of day was this" — one value, and it is what any future payroll logic reads. flags answer "what needs a human look" — zero or more, driving the exception list. Somebody can be PRESENT and also LATE_IN and SHORT_HOURS. One field would force values like PRESENT_BUT_LATE_AND_SHORT, which stops scaling at about three combinations.

Why the file is pure

No React, no fetch, no database. Three consequences: it is testable by passing an array in and asserting on what comes out; the mock server and the real backend produce identical results because this is the single definition; and in Phase 2 it is the one file that gets ported to Python while everything else stays as it is.

3.2 The permission system

src/contracts/permissions.ts · src/lib/rbac.ts · src/routes/ProtectedRoute.tsx

PERMISSIONS = ['employees.view_all', 'attendance.view_own', ...] as const
type Permission = (typeof PERMISSIONS)[number] ← a union, not `string`

login response → { user: { permissions: Permission[] } } flat, resolved

authStore.setSession → permissions: Set<Permission>

↓

Sidebar filters nav

↓

PermissionGate wraps a button

↓

ProtectedRoute guards a route

↓

feature pages branch on useCan()

#	Level	Security?	Implementation
1	Sidebar and buttons	No — UX	PermissionGate , sidebar filter
2	Route guard	No — UX	ProtectedRoute need={[[...]]}
3	API endpoint	Yes	Phase 2: FastAPI dependency per route
4	Row scope	Yes	WHERE employee_id = :me when only view_own
5	Field visibility	Yes	Serialiser omits restricted fields entirely
6	Audit trail	Forensic	Phase 2: every write logged before/after

LEVELS 1 AND 2 ARE NOT SECURITY

A hidden button does not stop anyone opening developer tools and calling /api/v1/employees/5 directly. Everything in lib/rbac.ts exists to stop people seeing controls they cannot use. The control is levels 3 and 4, on the server, on every endpoint including reads.

Why three view scopes rather than one "view" permission

view_all, view_team and view_own are distinct because the scope has to be derivable from the permission alone. With a single attendance.view you would end up writing if (role === 'manager') in the query layer — which breaks the first time somebody creates a custom role.

Why the login response carries a flat array

The client should never work out "this user has role X, role X contains Y and Z". That resolution has to happen server-side anyway for enforcement; doing it once means one implementation instead of two that can disagree.

3.3 Privilege-escalation guards on role change

src/mocks/handlers/users.ts → roleChangeError()

```
roleChangeError(actorPermissions, targetUserId, actorUserId, newRoleId)

1. actor lacks 'users.change_role'           → 403 FORBIDDEN
2. target is the actor                       → 403 SELF_ROLE_CHANGE
3. target's role has 'users.change_role'
   AND actor lacks 'roles.manage'           → 403 PEER_ROLE_CHANGE
4. newRole.permissions ∅ actorPermissions   → 403 ESCALATION
otherwise                                   → null (allowed)
```

Rule 4 does the real work: a manager cannot grant roles.manage because they do not hold it, so they cannot create an owner. Escalation becomes impossible without any hard-coded hierarchy of roles — no "owner > manager > employee" table to maintain.

The dropdown is filtered by the server too (GET /roles/assignable), but that is a convenience. The check on write is the control.

3.4 Idempotency on punches

src/domain/attendanceRules.ts → buildIdempotencyKey()

```
key = `${employeeId}:${yyyy-MM-dd}:${direction}:${HHmm}`

client:  writes punch locally, POSTs, retries on network failure
server:  key already present → 409 DUPLICATE_PUNCH
client:  treats 409 as SUCCESS and stops retrying
```

Minute-level granularity: two punches in the same minute are the same punch. This exists now, before the Phase 3 kiosk needs it, because adding the field after the punch table has history means backfilling a key for every existing row.

3.5 Session limits

src/lib/session.ts · useIdleTimer.ts · layouts/SessionGuard.tsx

Limit	Default	Resets on activity	Purpose
Idle	30 min	Yes, warns at 2 min left	The laptop somebody walked away from
Absolute	10 hours	No, no warning	A script nudging the mouse would otherwise keep a session alive forever

```
useIdleTimer({ enabled, onIdle, onExpired })

lastActivity : useRef ← mousemove fires ~60x/sec; state here would
sessionStart : useRef   re-render the entire tree on every pixel

listeners (passive): mousedown mousemove keydown wheel scroll
                     touchstart pointerdown + visibilitychange

setInterval 1s:
  now - sessionStart ≥ ABSOLUTE → onExpired()
  now - lastActivity ≥ IDLE     → onIdle()
  remaining ≤ WARNING          → setWarningMsLeft(remaining)
  otherwise                    → leave state untouched
```

On expiry, clearSession(reason) flips isAuthenticated to false. ProtectedRoute reads that, so every guarded route bounces to /login immediately and the sidebar goes with the shell. Nothing needs disabling by hand and no module is reachable by typing a URL. The login screen then names which limit was hit — without that, somebody who was working returns to a login form with no idea whether the app crashed.

3.6 Pagination envelope

```
list: { data: T[], meta: { page, pageSize, total, totalPages } }
single: { data: T }
error: { error: { code, message, fields?: { field: message } } }
```

Every list endpoint is paginated from day one, including ones that return twelve rows today. If the UI is built assuming a plain array, adding pagination later means rewriting every table, filter and query hook. DataTable uses manualPagination: true for the same reason: the server pages, so the same component works at twelve rows and at two thousand.

4. Complete file reference

84 TypeScript files, ~11,150 lines. Configuration files at the root are listed at the end.

4.1 [src/contracts/](#) — the frozen agreement

Read by every layer, changed by none of them. If a shape crosses the network, it is declared here and nowhere else.

File	Lines	Exports and purpose
types.ts	575	Every request and response shape. Branded IDs (<code>EmployeeId</code> , <code>TaskId</code> ... — strings that TypeScript refuses to interchange). <code>Paginated<T></code> , <code>Single<T></code> , <code>ApiError</code> . Domain types: <code>Employee</code> , <code>PunchEvent</code> , <code>AttendanceDay</code> , <code>AttendanceFlag</code> , <code>Task</code> , <code>Role</code> , <code>User</code> , <code>Shift</code> , <code>OrgSettings</code> , <code>DashboardStats</code> , <code>AttendanceTrendPoint</code> . Dates are ISO strings, never <code>Date</code> objects — JSON has no <code>Date</code> type, and a half-parsed mix is a reliable source of timezone bugs.
permissions.ts	156	<code>PERMISSIONS</code> (the <code>as const</code> array), <code>Permission</code> (the union derived from it), <code>PERMISSION_MODULES</code> (grouping for the matrix), <code>PERMISSION_LABELS</code> , <code>PERMISSION_DESCRIPTIONS</code> (plain English, shown on hover and in the guide).
endpoints.ts	88	<code>API_BASE</code> , <code>EP</code> (every URL, grouped), <code>EP_PATTERNS</code> (<code>:id</code> forms for MSW). Handlers register against the same constants, so a mock and its real endpoint cannot drift.

4.2 [src/domain/](#) — business rules

File	Lines	Functions
attendanceRules.ts	174	<code>buildDay()</code> punches + shift → one day record with status and flags · <code>isCurrentlyIn()</code> is the last punch of today an IN · <code>buildIdempotencyKey()</code> the duplicate-rejection key · <code>FLAG_LABELS</code> human labels for the UI.

4.3 [src/lib/](#) — infrastructure

File	Lines	Functions and why it exists
apiClient.ts	111	<code>apiClient.get/post/patch/put/delete</code> , <code>setAccessToken()</code> , <code>getAccessToken()</code> , <code>ApiException</code> (carries status, code and per-field errors). The only file that knows about fetch, base URLs and the token — forty components calling fetch would be forty chances to forget the header. The token lives in a module variable, not <code>localStorage</code> , so an XSS bug is not an account takeover.
rbac.ts	43	<code>can()</code> , <code>canAny()</code> for use outside React · <code>useCan()</code> , <code>useCanAny()</code> hooks that re-render when the user changes. The only place that decides what a user may see.
queryClient.ts	32	<code>queryClient</code> . 30s <code>staleTime</code> , refetch on window focus, one retry — but never on a 4xx, because repeating a request the server already rejected does not help.
format.ts	49	<code>formatTime()</code> , <code>formatDate()</code> , <code>formatDuration()</code> , <code>minutesSince()</code> , <code>initials()</code> , <code>todayISO()</code> . Every contract date is a string; it is parsed here at render time and nowhere else.
session.ts	55	<code>IDLE_LIMIT_MS</code> , <code>IDLE_WARNING_MS</code> , <code>ABSOLUTE_LIMIT_MS</code> , <code>ACTIVITY_EVENTS</code> , <code>LOGOUT_MESSAGES</code> , <code>LogoutReason</code> . Policy in one place; changing a limit means editing one number.
useIdleTimer.ts	109	<code>useIdleTimer()</code> → <code>{ warningMsLeft, resetIdle }</code> . Passive listeners, timestamps in refs, one interval per second.

4.4 [src/stores/](#) — global client state

File	Lines	Shape
authStore.ts	77	<code>useAuthStore</code> : <code>user</code> , <code>permissions</code> : <code>Set<Permission></code> , <code>isAuthenticated</code> , <code>logoutReason</code> , <code>setSession()</code> , <code>clearSession(reason)</code> , <code>acknowledgeLogout()</code> . A <code>Set</code> rather than an array because <code>can()</code> runs on nearly every render and membership is the actual question being asked.
uiStore.ts	56	<code>useUiStore</code> : <code>collapsed</code> (persisted — a deliberate desktop preference), <code>mobileOpen</code> (not persisted — nobody wants to land on a page with the menu already covering it), <code>toggleCollapsed()</code> , <code>setMobileOpen()</code> .

4.5 src/components/common/ — shared UI

Built once so screens cannot drift apart. Anything appearing on two screens belongs here.

File	Lines	Purpose
DataTable.tsx	163	The table behind every list screen. Server-side pagination, skeleton rows while loading (keeping the table's shape so the page does not jump), empty state, optional row click.
ConfirmDialog.tsx	144	ConfirmProvider + useConfirm() returning a promise. Replaces window.confirm and any alert library. On danger dialogs autoFocus sits on Cancel, so a reflexive Enter cancels rather than deletes.
Toast.tsx	79	ToastProvider + useToast(). Outcomes the user only needs told. aria-live so it is announced.
Modal.tsx	76	Centred dialog on Radix — focus trap, Escape, scroll lock, portal, aria-hidden on the rest of the page. A week of work to get right by hand, and screen-reader users notice immediately when it is wrong.
Drawer.tsx	64	Right-hand panel. Same Radix Dialog underneath — a drawer is a modal that slides from the edge, not a different mechanism.
SearchInput.tsx	58	300ms debounce. Without it, typing "engineering" fires twelve requests that can return out of order, so results for "engineer" overwrite results for "engineering".
PriorityBadge.tsx	53	Icon plus word. A bare coloured dot needs a legend, and roughly one man in twelve cannot reliably separate red from amber.
Avatar.tsx	60	Photo, or initials. The fallback has to exist everywhere an avatar does.
Select.tsx	54	Styled native <select>. Keyboard-accessible for free and opens the OS picker on mobile — the heavier Radix component is for cases the light one cannot do.
Button.tsx	48	primary · secondary · ghost · danger.
Chip.tsx / StatusPill.tsx	46 / 44	The shared pill recipe: 15% tint, 600 text, 20% border.
FormField.tsx	45	Label, control and error as one unit, plus inputCls(). Bundled so the error never lands somewhere unexpected and the label stays linked for screen readers.
EmptyState.tsx	42	An empty table with only headers reads as broken, not as "no results".
PageHeader.tsx	35	Title, description, actions. Built once so every page lines up.
PermissionGate.tsx	25	Renders children only if the permission is held.

4.6 src/layouts/ — the shell

File	Lines	Purpose
Sidebar.tsx	311	Permission-filtered navigation. Collapsible to a 72px icon rail on desktop, an off-canvas drawer below 1024px. Closes on navigate and on Escape, locks body scroll while open. Real headcount for those allowed to see it, own department for everyone else.
SessionGuard.tsx	102	Runs the idle timer for the whole signed-in app and renders the countdown warning. Escape and backdrop clicks are blocked — "does Escape mean keep me signed in, or sign me out?" has no obvious answer, so the choice is two explicit buttons.
AppShell.tsx	50	Sidebar + top bar + <Outlet />. Content offset follows the collapse state. Sits above pages in the route tree, so the sidebar never unmounts during navigation.
MobileTopBar.tsx	48	Hamburger, brand, avatar. Below lg only — above it the sidebar is already on screen and a second bar would waste the vertical space a laptop has least of.

4.7 src/routes/

File	Lines	Purpose
index.tsx	85	The route tree. All pages lazy except login; each wrapped in a Suspense fallback that matches the page's shape.
ProtectedRoute.tsx	38	Redirects to /login when unauthenticated, to /dashboard when the permission is missing. Uses replace so the back button does not bounce the user into the page they were just refused.

4.8 src/features/ — screens and their hooks

Each feature owns its pages and a single `api.ts` holding every query and mutation for that feature. Components never call `apiClient` directly.

auth/ — 218 lines

File	Contents
api.ts	<code>useLogin()</code> stores the session and navigates; <code>useLogout()</code> clears the session and the entire query cache in <code>onSettled</code> , so a failed logout request still logs the person out locally.
LoginPage.tsx	Form, error banner, demo accounts, and the amber notice naming which session limit ended the last session.

dashboard/ — 1,048 lines

File	Contents
api.ts	<code>useDashboardStats()</code> , <code>useExceptions()</code> , <code>useMyToday()</code> , <code>useAttendanceTrend(granularity)</code> . The first three refetch on a 60s interval, which keeps the screen current without the machinery of a websocket.
DashboardPage.tsx	Renders <code>MyDayStrip</code> for anyone attendance-tracked, then branches on <code>can('attendance.view_all')</code> — capability, not role name, so a custom role gets the right view with no code change.
MyDayStrip.tsx	Own check-in, check-out and hours. Shown to everyone tracked, including managers.
OwnerDashboard.tsx	Four counters, trend chart, live present table, exception panel, open tasks, plus the viewer's own hours chart if they are tracked.
EmployeeDashboard.tsx	Own tasks with add and complete, done-today list, 30-day hours chart with summary tiles.
TeamTrendChart.tsx	Stacked present/late/absent bars, daily · weekly · monthly. Absent sits at the top of the stack because a middle segment has no common baseline and cannot be compared across bars. Tooltip carries percentages — 8 of 100 and 8 of 20 are different stories.
MonthChart.tsx	Hours per day for one person, with a target reference line. Replaced a grid of unlabelled coloured squares that could only be read against a legend.
StatCard.tsx	One number plus its context line. "36" alone is meaningless; "36 of 40 tracked" answers the question it was asked for.

attendance/ — 651 lines

File	Contents
api.ts	<code>useAttendanceDays()</code> , <code>usePresentEmployees()</code> (30s interval), <code>usePunches()</code> (only fires when a row is expanded), <code>useCreatePunch()</code> .
AttendancePage.tsx	Register with date-range presets, summary strip, exceptions-only toggle, expandable rows, pagination. Scoped to own records for anyone without <code>view_all</code> .
PunchList.tsx	The raw events behind one day — time, direction, device, confidence, source. This is the audit trail made visible: "the kiosk matched your face at 91% at 09:14" is a very different conversation from "the system says you were late".
ManualPunchModal.tsx	Creates a new event with <code>source: 'manual'</code> . There is no edit path anywhere in the system.

employees/ — 885 lines

File	Contents
api.ts	<code>useEmployees()</code> , <code>useEmployee()</code> , <code>useDepartments()</code> , <code>useShifts()</code> (reference data, <code>staleTime: Infinity</code>), <code>useCreateEmployee()</code> , <code>useUpdateEmployee()</code> .
EmployeeListPage.tsx	Table with URL-backed filters, detail drawer, create/edit modal.
EmployeeDetailDrawer.tsx	Details, Attendance and Tasks tabs. The attendance tab reuses <code>useAttendanceDays</code> with an employee filter — the payoff of putting hooks in feature-level <code>api.ts</code> files rather than inside pages.
EmployeeForm.tsx	Create and edit in one modal. Zod validates in the browser for instant feedback; the server validates again because a form can be bypassed. Server 422 field errors are pushed back into the same display.

tasks/ — 691 lines

File	Contents
api.ts	<code>useTasks()</code> , <code>useCreateTask()</code> , <code>useUpdateTask()</code> , <code>useCompleteTask()</code> .
TasksPage.tsx	Three-column board. One route for both audiences — the owner assigns and sees everyone, an employee sees only their own and can only complete.
TaskCard.tsx	Title, priority badge, assignee, due date, and the actions available: Start, back to To do, Mark done, Reopen, Edit.
TaskForm.tsx	Three jobs, one component: assign to someone, add to my own list, edit an existing task. The fields are identical; three components would mean adding every future field three times.

roles/ — 383 lines · users/ — 886 lines · settings/ — 450 lines

File	Contents
roles/RolesPage.tsx	The matrix. Changes are staged locally and saved in one action — per-checkbox requests would leave a half-applied set on a dropped connection, and would fire the "last role that can manage roles" check against an intermediate state. The signed-in user's own permissions are deliberately not refreshed; silently removing your own menu items mid-session is disorienting.
users/UsersPage.tsx	Accounts table. Disabling is a toggle, never a delete — deleting orphans the audit trail, because attendance history has to keep pointing at somebody.
users/UserForm.tsx	Creates a login. The temporary password is generated in the browser, shown once with a copy button, and never returned by any endpoint afterwards.

users/ResetPasswordModal.tsx	Admin reset. Note what is missing: no "current password" field. That is the point — the person has forgotten it. The self-service change under Settings does require it.
users/ChangeRoleModal.tsx	Server-filtered role list plus a confirm step. Blocks self-change in the UI and again on the server.
settings/SettingsPage.tsx	Profile, Password, Organisation. The Organisation tab is hidden rather than disabled — a greyed-out tab tells an employee a setting exists and that they are not trusted with it, which invites a question somebody then has to answer. Policy changes are retroactive and say so, because days are derived from punches.

4.9 src/mocks/ — the fake server

Deleted at the Phase 2 cutover. Written as faithfully as a real server, because a mock that just returns the whole array lets the UI develop habits that break the day a real backend answers.

File	Lines	Endpoints / contents
browser.ts	58	<code>startMockServer()</code> . Unhandled <code>/api</code> requests throw rather than warn — Chrome hides warnings under "N hidden", which makes a missing handler look identical to a dead worker. Prints a handler count on start.
handlers/index.ts	29	Spreads all handler arrays into one.
handlers/auth.ts	134	<code>POST /auth/login</code> , <code>GET /me</code> , <code>POST /auth/logout</code> . <code>currentActor()</code> is the mock's stand-in for reading the actor from a bearer token. One generic message for wrong username and wrong password, so usernames cannot be enumerated.
handlers/employees.ts	166	List with filter/sort/paginate, detail, create with server-side validation, patch.
handlers/attendance.ts	246	Days (computed via <code>buildDay</code> on every request, never stored), present list, punches, manual punch with idempotency, my-today.
handlers/tasks.ts	166	List, create, patch, complete. <code>isOverdue</code> is recomputed on read, never persisted — a stored value is wrong the next morning. The server stamps <code>completedAt</code> , not the browser, whose clock may be wrong.
handlers/roles.ts	167	List, catalogue, create, patch, delete. Guards: system roles cannot be renamed or deleted; the last in-use role holding <code>roles.manage</code> cannot lose it; a role in use cannot be deleted.
handlers/users.ts	322	List, create, reset password (204 — nothing to return and nothing that could leak), status toggle with a last-admin guard, role change with the four escalation rules, assignable-roles.
handlers/dashboard.ts	231	Stats, exceptions, and the trend aggregation. Aggregated server-side: six months of day-level rows for a growing company is thousands of records to answer a question about twenty numbers.
handlers/settings.ts	161	Org settings read and patch, own profile, own password change (which does require the current one).
fixtures/employees.ts	238	Twelve people. Deliberately uneven: two without logins, one inactive, one with no face enrolled, one not attendance-tracked. Clean fixtures make a demo look good and hide every edge case until production.
fixtures/punches.ts	179	Fourteen days, generated from a seeded PRNG so the demo is identical on every reload — <code>Math.random</code> would reshuffle who was late and make a reported bug unreproducible. Per-person behaviour profiles produce chronic lateness, forgotten check-outs and absences. Regenerates when the date rolls over, so a tab left open overnight does not read as zero everywhere.
fixtures/tasks.ts	178	Nine tasks: two overdue, one completed today, one with no due date, several people with nothing.
fixtures/roles.ts	80	Owner, Manager, Employee with their permission sets. Mutable, because the matrix edits them.
fixtures/users.ts	70	Three accounts. Passwords are plain text here only because this runs entirely in the browser; the real backend stores an argon2 hash and never the password.
fixtures/org.ts	26	Four departments, one company-wide shift.

4.10 Root configuration

File	Purpose
tailwind.config.js	The shared design system: Inter, #FAF9F6 canvas, indigo #4F46E5 primary, zinc text ramp, radius scale 4/8/12/16, shadow scale, spacing tokens, type tokens.
src/index.css	Font imports, Material Symbols, pointer cursor and transitions applied once for all controls, focus ring, 5px scrollbars, tabular numerals. No <code>@apply</code> of custom colours in the base layer — that resolves against the config at build time and fails outright if the token is not loaded.
vite.config.ts	<code>@</code> path alias to <code>src</code> .
tsconfig.app.json	Path mapping mirroring Vite. <code>strict: false</code> for now — turn it on once fluent and fix what it reports in a day.
public/mockServiceWorker.js	Generated by <code>npm msw init</code> . Must match the installed msw version.

5. Conventions — the rules that keep it coherent

Rule	Why
Components never call <code>fetch</code> or <code>ApiClient</code>	They call a hook from <code>features/<name>/api.ts</code> . Otherwise the Phase 2 cutover means visiting every component.
Network shapes live only in <code>contracts/types.ts</code>	An inline <code>{ id: string; name: string }</code> in a component is a shape that will silently diverge from the server.
Permission strings are never written inline	Import from <code>contracts/permissions.ts</code> , so a typo is a compile error rather than a silent <code>false</code> .
Branch on capability, never on role name	<code>can('attendance.view_all')</code> , not <code>role === 'owner'</code> . A custom role then works with no code change.
Filters go in the URL	Back button, shareable links, refresh-safe.
Every list endpoint is paginated	Retrofitting means rewriting every table.
Punch events are append-only	Corrections are new rows. The first salary dispute needs evidence, and edited rows are not evidence.
Time-relative values are computed on read	<code>isOverdue</code> stored today is wrong tomorrow.
The server stamps times, not the browser	A device clock can be wrong or in another timezone.
No <code>window.confirm</code> , no alert library	<code>useConfirm()</code> and <code>useToast()</code> — one dialog mechanism, one set of tokens.
Colour is reserved for status	A department is neither good nor bad. Colour that decorates drowns out colour that warns.
Cross-feature invalidation is explicit	Creating a login changes users, employees and roles. Forgetting one is the most common "why is it showing the old value" bug.

6. Phase 2 migration map

Phase 1	Phase 2	Effort
<code>domain/attendanceRules.ts</code>	<code>domain/attendance_rules.py</code>	Direct port. Pure functions, no dependencies. The only file carrying business logic.
<code>mocks/handlers/*.ts</code>	FastAPI routers	Same URLs, same query params, same envelope. The handlers are the specification.
<code>mocks/fixtures/*.ts</code>	Seed script + Alembic migrations	The fixture shapes are the table columns.
Dashboard trend loop	<code>GROUP BY date_trunc(...)</code>	Rewrite as SQL — considerably faster than the JS loop.
In-memory arrays	PostgreSQL + pgvector	pgvector for Phase 3 face embeddings.
Mock JWT string	Real JWT: 15-min access, 7-day refresh in an <code>httpOnly</code> cookie	<code>ApiClient</code> gains a <code>refresh-on-401</code> path. Nothing else changes.
Client-only permission checks	Per-endpoint dependency + row scoping in SQL	The essential work. Everything in Phase 1 was UX.
<code>enableMocking()</code> in <code>main.tsx</code>	Deleted	One line. No component, hook or URL changes.

WHAT PHASE 1 ACTUALLY DELIVERED

Not just screens. A frozen API contract, a single definition of the attendance rules, a permission model that scales past its two seeded roles, and a mock server faithful enough that the real one can be written against it. The backend has a specification to implement rather than a blank page.

CARRIED FORWARD AS OPEN ITEMS

Leave and holidays — without them, somebody on approved leave and somebody who did not turn up are indistinguishable, and every absence figure is wrong from the first month. This belongs in the Phase 2 schema, not bolted on later.

Server-side enforcement — every permission check in Phase 1 is user experience. None of it is security until Phase 2 re-checks it per endpoint, including reads, with row-level scoping in SQL.

Biometric consent — employee face templates are personal data processed by an employer. Written consent, a defined retention period, embeddings stored rather than photographs, and a PIN fallback for anyone who declines. Requires legal review before Phase 3.